

Subject Code	:	CEUE203	Semester	:	3	Academic Year	:	2026-27
Subject Name	:	Object-Oriented Programming						

	Practical 1: CLASS FUNDAMENTALS
Program: 1.1	Vending machine (VendingMachine.java, main in this file) (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total – 15). Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

Code

```
import java.util.Scanner;

public class VendingMachine {
    enum Coin {
        ONE, TWO, FIVE, TEN
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int price = 15;
        int total = 0;

        while (total < price) {
            Coin coin = Coin.valueOf(sc.next().toUpperCase());

            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };

            total += value;
            System.out.println("Total: " + total);
        }

        System.out.println("Paid. Change: " + (total - price));
        sc.close();
    }
}
```

Output	TEN Total: 10 FIVE Total: 15 Paid. Change: 0
---------------	--

Program: 1.2	Toll booth (TollBooth.java, main in this file) (a) Define a record Vehicle(String number, String type). (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter. (d) After the loop, print the total toll and the type with the highest count. Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.
---------------------	--

Code	<pre>import java.util.Scanner; public class TollBooth { record Vehicle(String number, String type) {} public static void main(String[] args) { Scanner sc = new Scanner(System.in); int total = 0, bike = 0, car = 0, truck = 0; while (true) { System.out.print("Vehicle number: "); String number = sc.next(); if (number.equalsIgnoreCase("done")) { break; } System.out.print("Vehicle type: "); String type = sc.next().toLowerCase(); Vehicle v = new Vehicle(number, type); int toll = switch (v.type()) { case "bike" -> 20; case "car" -> 50; case "truck" -> 150; default -> 0; }; if (toll == 0) { System.out.println("Invalid vehicle type"); continue; } total += toll; switch (v.type()) { case "bike" -> bike++; case "car" -> car++; case "truck" -> truck++; } } System.out.println("Total toll: " + total); if (bike >= car && bike >= truck) { System.out.println("Most frequent: bike"); } else if (car >= bike && car >= truck) { System.out.println("Most frequent: car"); } else { System.out.println("Most frequent: truck"); } } }</pre>
-------------	--

```
    }  
    sc.close();  
  }  
}
```

Output	Vehicle number: GJ01 Vehicle type: car Vehicle number: GJ02 Vehicle type: car Vehicle number: GJ03 Vehicle type: bike Vehicle number: done Total toll: 120 Most frequent: car
---------------	---

Program: 1.3	(For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java) (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player’s Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner. Expected output: each round prints both moves and the round winner; ends with a “You win 3–2” summary.
---------------------	--

Code

```
import java.util.Scanner;
import java.util.Random;

public class RPSLS {
    enum Move {
        ROCK, PAPER, SCISSORS, LIZARD, SPOCK
    }

    static int winner(Move a, Move b) {
        if (a == b) {
            return 0;
        }

        boolean win = switch (a) {
            case ROCK -> b == Move.SCISSORS || b == Move.LIZARD;
            case PAPER -> b == Move.ROCK || b == Move.SPOCK;
            case SCISSORS -> b == Move.PAPER || b == Move.LIZARD;
            case LIZARD -> b == Move.PAPER || b == Move.SPOCK;
            case SPOCK -> b == Move.ROCK || b == Move.SCISSORS;
        };

        return win ? 1 : -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        int playerScore = 0, computerScore = 0;

        Move[] moves = Move.values();

        for (int i = 1; i <= 5; i++) {
            System.out.print("Enter move: ");
            Move player = Move.valueOf(sc.next().toUpperCase());

            Move computer = moves[random.nextInt(moves.length)];

            System.out.println("You: " + player);
            System.out.println("Computer: " + computer);

            int result = winner(player, computer);

            if (result == 1) {
                System.out.println("You win this round");
                playerScore++;
            } else if (result == -1) {
                System.out.println("Computer wins this round");
                computerScore++;
            } else {
                System.out.println("Tie");
            }
        }
    }
}
```

```
}

System.out.println("You: " + playerScore);
System.out.println("Computer: " + computerScore);

if (playerScore > computerScore) {
    System.out.println("You win");
} else if (computerScore > playerScore) {
    System.out.println("Computer wins");
} else {
    System.out.println("Match tied");
}

sc.close();
}
```

Output	Enter move: rock You: ROCK Computer: SCISSORS You win this round Enter move: paper You: PAPER Computer: ROCK You win this round Enter move: scissors You: SCISSORS Computer: LIZARD You win this round Enter move: rock You: ROCK Computer: SPOCK Computer wins this round Enter move: paper You: PAPER Computer: SCISSORS Computer wins this round You: 3 Computer: 2 You win
---------------	---

Program: 1.4	Project Work: MiniBank — Menu Shell MiniBank is the console banking application you will build across the whole semester. In this first lab you set up your tools and create the project and its menu shell — the screen the user navigates. Every later lab attaches one real banking feature behind these menu options. 1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator. 2. Create a public class named MiniBank that contains the main method. 3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header. 4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer. 5. Display the numbered menu to the user and read the chosen number with a Scanner. 6. Use a switch expression on the choice to print a short placeholder for that option (for example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message. Expected output: The header and menu are shown; entering 2 prints “Deposit — to be implemented in a later lab”; entering the EXIT number prints a goodbye message and stops the program.
---------------------	---

Code	<pre> import java.util.Scanner; public class MiniBank { record BankInfo(String name, String branch) {} enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT } public static void main(String[] args) { Scanner sc = new Scanner(System.in); BankInfo bank = new BankInfo("MiniBank", "Main Branch"); System.out.println(bank.name() + " - " + bank.branch()); int choice; do { System.out.println("1. Open Account"); System.out.println("2. Deposit"); System.out.println("3. Withdraw"); System.out.println("4. Transfer"); System.out.println("5. Exit"); System.out.print("Enter choice: "); choice = sc.nextInt(); if (choice == 5) { System.out.println("Goodbye!"); break; } MenuOption option = switch (choice) { case 1 -> MenuOption.OPEN_ACCOUNT; case 2 -> MenuOption.DEPOSIT; case 3 -> MenuOption.WITHDRAW; case 4 -> MenuOption.TRANSFER; default -> null; }; if (option == null) { System.out.println("Invalid choice"); } else { switch (option) { case OPEN_ACCOUNT -> System.out.println("Open Account - to be implemented in a later lab"); case DEPOSIT -> System.out.println("Deposit - to be implemented in a later lab"); case WITHDRAW -> System.out.println("Withdraw - to be implemented in a later lab"); case TRANSFER -> </pre>
------	---

```
        System.out.println("Transfer - to be implemented in a later lab");
        case EXIT -> System.out.println("Goodbye!");
    }
}
} while (true);

sc.close();
}
}
```

Output	MiniBank - Main Branch 1. Open Account 2. Deposit 3. Withdraw 4. Transfer 5. Exit Enter choice: 2 Deposit - to be implemented in a later lab 1. Open Account 2. Deposit 3. Withdraw 4. Transfer 5. Exit Enter choice: 5 Goodbye!
---------------	---

	Practical 2: CLASSES, OBJECTS AND ENCAPSULATION
Program: 2.1	Smart thermostat (Thermostat.java, main in this file) (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)". (e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)". (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). Expected output: temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.

Code	<pre>public class Thermostat { private String location; private int temperature; private static final int MIN = 16; private static final int MAX = 30; private static int activeCount = 0; Thermostat(String location, int startTemp) { this.location = location; if (startTemp >= MIN && startTemp <= MAX) { temperature = startTemp; } else { temperature = 22; } activeCount++; } Thermostat(String location) { this(location, 22); } void raise() { if (temperature < MAX) { temperature++; } else { System.out.println("Already at maximum (30)"); } } void lower() { if (temperature > MIN) { temperature--; } else { System.out.println("Already at minimum (16)"); } } int getTemperature() { return temperature; } static int getActiveCount() { return activeCount; } public static void main(String[] args) { Thermostat t1 = new Thermostat("Room 1", 25); Thermostat t2 = new Thermostat("Room 2"); for (int i = 0; i < 10; i++) { t1.raise();</pre>
-------------	--

	<pre> System.out.println(t1.getTemperature()); } for (int i = 0; i < 20; i++) { t1.lower(); System.out.println(t1.getTemperature()); } System.out.println("Active thermostats: " + getActiveCount()); } }</pre>
--	---

Output	26 27 28 29 30 Already at maximum (30) 30 Already at maximum (30) 30 Already at maximum (30) 30 Already at maximum (30) 30 Already at maximum (30) 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 Already at minimum (16) 16 Already at minimum (16) 16 Already at minimum (16) 16 Already at minimum (16) 16 Already at minimum (16) 16 Already at minimum (16) 16 Active thermostats: 2
--------	--

Program: 2.2	Cinema show (CinemaShow.java, main in this file) (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if $n \leq \text{seatsAvailable}$, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked(). Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.
---------------------	---

Code	<pre>public class CinemaShow { private String title; private int seatsAvailable; private final int capacity; private static int totalBooked = 0; CinemaShow(String title, int capacity) { this.title = title; this.capacity = capacity; seatsAvailable = capacity; } CinemaShow(String title) { this(title, 100); } boolean book(int n) { if (n > 0 && n <= seatsAvailable) { seatsAvailable -= n; totalBooked += n; return true; } return false; } void cancel(int n) { if (n > 0) { seatsAvailable = Math.min(capacity, seatsAvailable + n); } } int getSeatsAvailable() { return seatsAvailable; } static int getTotalBooked() { return totalBooked; } public static void main(String[] args) { CinemaShow show = new CinemaShow("Avengers", 50); System.out.println("Booking 20: " + show.book(20)); System.out.println("Seats available: " + show.getSeatsAvailable()); System.out.println("Booking 40: " + show.book(40)); System.out.println("Seats available: " + show.getSeatsAvailable()); System.out.println("Booking 10: " + show.book(10)); System.out.println("Seats available: " + show.getSeatsAvailable()); } }</pre>
-------------	---

	<pre>show.cancel(5); System.out.println("After cancellation: " + show.getSeatsAvailable()); System.out.println("Total booked: " + CinemaShow.getTotalBooked()); } }</pre>
--	--

Output	Booking 20: true Seats available: 30 Booking 40: false Seats available: 30 Booking 10: true Seats available: 20 After cancellation: 25 Total booked: 30
---------------	--

Program: 2.3	(For Advanced Learners – home) Parking lot (ParkingLot.java) (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue. Expected output: parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).
---------------------	---

Code

```
public class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
    }

    void park(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                System.out.println("Two-wheeler parked");
            } else {
                System.out.println("Full");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                System.out.println("Four-wheeler parked");
            } else {
                System.out.println("Full");
            }
        }
    }

    void leave(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left");
            }
        }
    }

    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 1);

        lot.park("two");
        lot.park("two");
    }
}
```

	<pre>lot.park("two"); lot.park("four"); lot.park("four"); lot.leave("two"); System.out.println("Two-wheelers: " + lot.twoWheelers); System.out.println("Four-wheelers: " + lot.fourWheelers); System.out.println("Revenue: " + revenue); } }</pre>
--	--

Output	Two-wheeler parked Two-wheeler parked Full Four-wheeler parked Full Two-wheeler left Two-wheelers: 1 Four-wheelers: 1 Revenue: 80
---------------	---

Program: 2.4	Project Work: MiniBank — Customer and Account Classes In Practical 1 you built the empty menu shell. Now you create the data the bank works on — the Customer and Account classes. Using encapsulation, an account's balance can change only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends. 1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as "CUST101". Add a constructor and public getter methods. 2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001"). 3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance. 5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance. Expected output: Balance prints as 7000 after the deposit, 4000 after the successful withdraw, and remains 4000 after the failed withdraw (which returns false).
---------------------	--

Code	<pre>class Customer { private String name; private String email; private String mobile; private final String customerId; private static long customerCounter = 100; private static String generateCustomerId() { customerCounter++; return "CUST" + customerCounter; } Customer(String name, String email, String mobile) { this.name = name; this.email = email; this.mobile = mobile; this.customerId = generateCustomerId(); } String getName() { return name; } String getEmail() { return email; } String getMobile() { return mobile; } String getCustomerId() { return customerId; } } public class Account { private final String accountNumber; private String ownerName; private long balance; private boolean active; private static int accountCounter = 0; Account(String ownerName, long balance) { this.ownerName = ownerName; this.balance = balance; this.active = true; accountCounter++; accountNumber = String.format("AC%04d", accountCounter); } }</pre>
-------------	--

```
Account(String ownerName) {
    this(ownerName, 0);
}

void deposit(long amount) {
    if (amount > 0 && active) {
        balance += amount;
    }
}

boolean withdraw(long amount) {
    if (amount > 0 && amount <= balance && active) {
        balance -= amount;
        return true;
    }
    return false;
}

String getAccountNumber() {
    return accountNumber;
}

String getOwnerName() {
    return ownerName;
}

long getBalance() {
    return balance;
}

boolean isActive() {
    return active;
}

public static void main(String[] args) {
    Customer c1 = new Customer("Rahul", "rahul@gmail.com", "9876543210");

    Account[] accounts = {
        new Account(c1.getName(), 5000),
        new Account("Amit", 3000),
        new Account("Neha")
    };

    accounts[0].deposit(2000);
    accounts[0].withdraw(1000);
    accounts[1].withdraw(4000);
    accounts[1].deposit(500);
    accounts[2].deposit(1500);

    System.out.println("Customer ID: " + c1.getCustomerId());

    for (Account account : accounts) {
```

	<pre>System.out.println(account.getAccountNumber() + " " + account.getOwnerName() + " Balance: " + account.getBalance()); } } }</pre>
--	---

Output	Customer ID: CUST101 AC0001 Rahul Balance: 6000 AC0002 Amit Balance: 3500 AC0003 Neha Balance: 1500
---------------	--

	Practical 3: OBJECT CLASS METHODS AND NESTED CLASSES
Program: 3.1	Distinct points (Point.java + Driver.java with main) (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return "(x, y)". (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print "Distinct: N". Expected output: 5 points with 2 repeats → "Distinct: 3".

Code	<pre>import java.util.Objects; class Point { private int x; private int y; Point(int x, int y) { this.x = x; this.y = y; } public String toString() { return "(" + x + ", " + y + ")"; } public boolean equals(Object obj) { if (this == obj) { return true; } if (!(obj instanceof Point)) { return false; } Point p = (Point) obj; return x == p.x && y == p.y; } public int hashCode() { return Objects.hash(x, y); } } public class Driver { public static void main(String[] args) { Point[] points = { new Point(1, 2), new Point(3, 4), new Point(1, 2), new Point(5, 6), new Point(3, 4) }; int distinct = 0; for (int i = 0; i < points.length; i++) { boolean found = false; for (int j = 0; j < i; j++) { if (points[i].equals(points[j])) { found = true; break; } } } } }</pre>
-------------	---

```
    }  
  
    if (!found) {  
        distinct++;  
    }  
}  
  
System.out.println("Distinct: " + distinct);  
}  
}
```

Output	Distinct: 3
---------------	-------------

Program: 3.2	Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears. Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".
---------------------	--

Code	<pre>import java.util.Objects; class Card { private String rank; private String suit; Card(String rank, String suit) { this.rank = rank; this.suit = suit; } public String toString() { return rank + " of " + suit; } public boolean equals(Object obj) { if (this == obj) { return true; } if (!(obj instanceof Card)) { return false; } Card c = (Card) obj; return rank.equals(c.rank) && suit.equals(c.suit); } public int hashCode() { return Objects.hash(rank, suit); } } public class Driver { public static void main(String[] args) { Card[] cards = { new Card("King", "Hearts"), new Card("Queen", "Spades"), new Card("Ace", "Clubs"), new Card("King", "Hearts"), new Card("Jack", "Diamonds") }; for (int i = 0; i < cards.length; i++) { for (int j = 0; j < i; j++) { if (cards[i].equals(cards[j])) { System.out.println("Duplicate found: " + cards[i]); return; } } } System.out.println("No duplicates found"); } }</pre>
-------------	---

	<pre>} }</pre>
--	--------------------

Output	Duplicate found: King of Hearts
---------------	---------------------------------

Program: 3.3	(For Advanced Learners – home) Fractions (Fraction.java + Driver.java) (a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute $g = \text{gcd}(\text{num}, \text{den})$ and divide both by g . (c) Override toString() to return num + "/" + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same. Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).
---------------------	---

Code	<pre>import java.util.Objects; class Fraction { private int num, den; public Fraction(int num, int den) { int g = gcd(Math.abs(num), Math.abs(den)); this.num = num / g; this.den = den / g; } private int gcd(int a, int b) { while (b != 0) { int temp = b; b = a % b; a = temp; } return a; } public String toString() { return num + "/" + den; } public boolean equals(Object o) { if (!(o instanceof Fraction)) { return false; } Fraction f = (Fraction) o; return num == f.num && den == f.den; } public int hashCode() { return Objects.hash(num, den); } } public class Driver { public static void main(String[] args) { Fraction f1 = new Fraction(1, 2); Fraction f2 = new Fraction(2, 4); Fraction f3 = new Fraction(3, 6); System.out.println(f1); System.out.println(f2); System.out.println(f3); System.out.println(f1.equals(f2)); System.out.println(f2.equals(f3)); } }</pre>
-------------	---

Output	1/2 1/2 1/2 true true
---------------	--

Program: 3.4	Project Work: MiniBank — toString, equals/hashCode and Nested Class In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it. 1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type. Expected output: Printing a1 shows a readable line such as "AC0001 Riya 4000"; a1.equals(a2) prints true; the instanceof check prints true for an Account.
---------------------	---

Code	<pre>import java.util.Objects; class Account { private final String accountNumber; private String ownerName; private long balance; private static int counter = 1; public Account(String ownerName, long balance) { this.accountNumber = String.format("AC%04d", counter++); this.ownerName = ownerName; this.balance = balance; } public String getAccountNumber() { return accountNumber; } public String getOwnerName() { return ownerName; } public long getBalance() { return balance; } public String toString() { return accountNumber + " " + ownerName + " " + balance; } public boolean equals(Object o) { if (!(o instanceof Account)) { return false; } Account a = (Account) o; return accountNumber.equals(a.accountNumber); } public int hashCode() { return Objects.hash(accountNumber); } } class Customer implements Cloneable { private String name; private String email; private String mobile; private Address address; public Customer(String name, String email, String mobile, Address address) { this.name = name; } }</pre>
-------------	--

```
this.email = email;
this.mobile = mobile;
this.address = address;
}

public Address getAddress() {
    return address;
}

public Customer clone() {
    try {
        return (Customer) super.clone();
    } catch (CloneNotSupportedException e) {
        throw new RuntimeException(e);
    }
}

public static class Address {
    private String line;
    private String city;
    private String pincode;

    public Address(String line, String city, String pincode) {
        this.line = line;
        this.city = city;
        this.pincode = pincode;
    }

    public String getLine() {
        return line;
    }

    public String getCity() {
        return city;
    }

    public String getPincode() {
        return pincode;
    }
}

public class MiniBank {
    public static void main(String[] args) {
        Account a1 = new Account("Riya", 4000);
        Account a2 = a1;

        System.out.println(a1);
        System.out.println(a1.equals(a2));
        System.out.println(a1 instanceof Account);

        Customer.Address address =
```

```
        new Customer.Address("Main Road", "Surat", "395001");

    Customer c1 =
        new Customer("Riya", "riya@example.com", "9876543210", address);

    Customer c2 = c1.clone();

    System.out.println(c2.getAddress().getCity());
    }
}
```

Output	AC0001 Riya 4000 true true Surat
---------------	---

	Practical 4: STRINGS AND REGULAR EXPRESSIONS
Program: 4.1	Password strength (PasswordChecker.java + Driver.java) (a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. “.*[A-Z].*”) or String methods. (b) Write strength(String pw) that counts how many rules pass and returns “Weak” (0–1), “Medium” (2–3) or “Strong” (4). (c) In Driver.main: test several passwords; for each, print which rules passed and the final label. Expected output: “abc” → Weak; “Abcd1234!” → Strong.

Code

```
class PasswordChecker {
    public static String strength(String pw) {
        int count = 0;

        if (pw.length() >= 8) count++;
        if (pw.matches(".*[A-Z].*")) count++;
        if (pw.matches(".*[0-9].*")) count++;
        if (pw.matches(".*[^a-zA-Z0-9].*")) count++;

        if (count <= 1) return "Weak";
        if (count <= 3) return "Medium";
        return "Strong";
    }

    public static void check(String pw) {
        System.out.println("Password: " + pw);
        System.out.println("Length >= 8: " + (pw.length() >= 8));
        System.out.println("Uppercase: " + pw.matches(".*[A-Z].*"));
        System.out.println("Digit: " + pw.matches(".*[0-9].*"));
        System.out.println("Special character: " + pw.matches(".*[^a-zA-Z0-9].*"));
        System.out.println("Strength: " + strength(pw));
        System.out.println();
    }
}

public class Driver {
    public static void main(String[] args) {
        PasswordChecker.check("abc");
        PasswordChecker.check("Abcd1234!");
    }
}
```

Output	Password: abc Length >= 8: false Uppercase: false Digit: false Special character: false Strength: Weak Password: Abcd1234! Length >= 8: true Uppercase: true Digit: true Special character: true Strength: Strong
---------------	---

Program: 4.2	Chat-log filter (ChatFilter.java + Driver.java) (a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts). (b) Read a keyword from the user. (c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts. (d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder. (e) Print "Matches: N" and the built report. Expected output: keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.
---------------------	---

Code	<pre>import java.util.Scanner; public class Driver { public static void main(String[] args) { Scanner sc = new Scanner(System.in); String[] logs = { "10:05 alice Hello there", "10:10 bob Good morning", "WrongLine" }; System.out.print("Enter keyword: "); String keyword = sc.nextLine(); StringBuilder report = new StringBuilder(); int count = 0; for (String line : logs) { String[] parts = line.split(" ", 3); if (parts.length < 3) { continue; } String time = parts[0]; String user = parts[1]; String message = parts[2]; if (message.toLowerCase().contains(keyword.toLowerCase())) { count++; report.append(time) .append(" ") .append(user) .append(": ") .append(message) .append("\n"); } } System.out.println("Matches: " + count); System.out.print(report); sc.close(); } }</pre>
-------------	--

Output	Enter keyword: hello Matches: 1 10:05 alice: Hello there
---------------	--

Program: 4.3	(For Advanced Learners – home) Template filler (TemplateFiller.java + Driver.java) (a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied). (b) Compile a Pattern for <code>\{(\w+)\}</code> and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found. (d) Print the filled template. Expected output: the unsupplied {date} becomes [?].
---------------------	--

Code

```
import java.util.regex.*;

public class Driver {
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};

        Pattern pattern = Pattern.compile("\\{(?:\\w+)}\\");
        Matcher matcher = pattern.matcher(template);
        StringBuilder result = new StringBuilder();
        int last = 0;

        while (matcher.find()) {
            result.append(template, last, matcher.start());

            String key = matcher.group(1);
            String value = "[?]";

            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(key)) {
                    value = values[i];
                    break;
                }
            }

            result.append(value);
            last = matcher.end();
        }

        result.append(template.substring(last));
        System.out.println(result);
    }
}
```

Output	Dear Riya, order A07 ships [?].
---------------	---------------------------------

Program: 4.4	Project Work: MiniBank — Validation and Command Parsing Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer. 1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean. 2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount. 3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object. 4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string. 5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts. Expected output: isValidMobile("9876543210") prints true and isValidMobile("12345") prints false; the parsed command prints type=DEPOSIT, accountNumber=AC0001, amount=500.
---------------------	---

Code	<pre> import java.util.regex.Pattern; class Validator { private static final Pattern MOBILE = Pattern.compile("[6-9][0-9]{9}"); private static final Pattern EMAIL = Pattern.compile("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"); private static final Pattern PAN = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]"); private static final Pattern IFSC = Pattern.compile("[A-Z]{4}0[A-Z0-9]{6}"); public static boolean isValidMobile(String s) { return MOBILE.matcher(s).matches(); } public static boolean isValidEmail(String s) { return EMAIL.matcher(s).matches(); } public static boolean isValidPan(String s) { return PAN.matcher(s).matches(); } public static boolean isValidIfsc(String s) { return IFSC.matcher(s).matches(); } } enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER } record Command(TransactionType type, String accountNumber, long amount) {} class CommandParser { public static Command parse(String line) { String[] parts = line.trim().split("\\s+"); if (parts.length != 3) { throw new IllegalArgumentException("Invalid command"); } TransactionType type = TransactionType.valueOf(parts[0]); String accountNumber = parts[1]; long amount = Long.parseLong(parts[2]); return new Command(type, accountNumber, amount); } } </pre>
------	--

```
class Account {
    String accountNumber;
    String ownerName;
    long balance;

    Account(String accountNumber, String ownerName, long balance) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
    }
}

class StatementFormatter {
    public static String buildStatement(Account account) {
        StringBuilder statement = new StringBuilder();
        statement.append("Account Number: ").append(account.accountNumber);
        statement.append("\nOwner: ").append(account.ownerName);
        statement.append("\nBalance: ").append(account.balance);
        return statement.toString();
    }
}

public class MiniBank {
    public static void main(String[] args) {
        System.out.println(Validator.isValidMobile("9876543210"));
        System.out.println(Validator.isValidMobile("12345"));
        System.out.println(Validator.isValidEmail("riya@gmail.com"));
        System.out.println(Validator.isValidEmail("riya@gmail"));
        System.out.println(Validator.isValidPan("ABCDE1234F"));
        System.out.println(Validator.isValidIfsc("SBIN0001234"));

        Command command = CommandParser.parse("DEPOSIT AC0001 500");
        System.out.println("type=" + command.type()
            + ", accountNumber=" + command.accountNumber()
            + ", amount=" + command.amount());

        Account account = new Account("AC0001", "Riya", 4000);
        System.out.println(StatementFormatter.buildStatement(account));
    }
}
```

Output	<pre>true false true false true true type=DEPOSIT, accountNumber=AC0001, amount=500 Account Number: AC0001 Owner: Riya Balance: 4000</pre>
---------------	--

	Practical 5: INHERITANCE AND POLYMORPHISM
Program: 5.1	Shape areas An abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.

Code	<pre>abstract class Shape { abstract double area(); } class Circle extends Shape { double radius; Circle(double radius) { this.radius = radius; } double area() { return Math.PI * radius * radius; } } class Rectangle extends Shape { double length, width; Rectangle(double length, double width) { this.length = length; this.width = width; } double area() { return length * width; } } class Triangle extends Shape { double base, height; Triangle(double base, double height) { this.base = base; this.height = height; } double area() { return 0.5 * base * height; } } public class ShapeDemo { public static void main(String[] args) { Shape[] shapes = { new Circle(2), new Rectangle(4, 5), new Triangle(6, 3) }; double total = 0;</pre>
------	--

```
double largest = 0;

for (Shape shape : shapes) {
    double a = shape.area();
    System.out.printf("%s area: %.2f%n",
        shape.getClass().getSimpleName(), a);

    total += a;

    if (a > largest) {
        largest = a;
    }

    System.out.printf("Running total: %.2f%n", total);
}

System.out.printf("Total area: %.2f%n", total);
System.out.printf("Largest area: %.2f%n", largest);
}
```

Output	Circle area: 12.57 Running total: 12.57 Rectangle area: 20.00 Running total: 32.57 Triangle area: 9.00 Running total: 41.57 Total area: 41.57 Largest area: 20.00
---------------	--

Program: 5.2	Payroll An abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.
---------------------	--

Code	<pre>abstract class Employee { String name; int id; Employee(String name, int id) { this.name = name; this.id = id; } abstract double monthlySalary(); } class FullTime extends Employee { double salary; FullTime(String name, int id, double salary) { super(name, id); this.salary = salary; } double monthlySalary() { return salary; } } class PartTime extends Employee { int hours; double rate; PartTime(String name, int id, int hours, double rate) { super(name, id); this.hours = hours; this.rate = rate; } double monthlySalary() { return hours * rate; } } class Intern extends Employee { double stipend; Intern(String name, int id, double stipend) { super(name, id); this.stipend = stipend; } double monthlySalary() { return stipend; } }</pre>
-------------	---

```
}  
  
public class Payroll {  
    public static void main(String[] args) {  
        Employee[] employees = {  
            new FullTime("Amit", 101, 50000),  
            new PartTime("Riya", 102, 80, 200),  
            new Intern("Jay", 103, 8000)  
        };  
  
        double total = 0;  
  
        for (Employee employee : employees) {  
            double salary = employee.monthlySalary();  
  
            System.out.println(employee.name + " (" + employee.id  
                + "): " + salary);  
  
            if (employee instanceof Intern) {  
                System.out.println("Intern employee");  
            }  
  
            total += salary;  
        }  
  
        System.out.println("Total payroll: " + total);  
    }  
}
```

Output	Amit (101): 50000.0 Riya (102): 16000.0 Jay (103): 8000.0 Intern employee Total payroll: 74000.0
---------------	--

Program: 5.3	(For Advanced Learners – home) Late-fee media Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.
---------------------	--

Code	<pre>abstract class Media { String title; int lateDays; Media(String title, int lateDays) { this.title = title; this.lateDays = lateDays; } abstract double lateFee(); } class Book extends Media { Book(String title, int lateDays) { super(title, lateDays); } double lateFee() { return lateDays * 2.0; } } class DVD extends Media { DVD(String title, int lateDays) { super(title, lateDays); } double lateFee() { return lateDays * 5.0; } } class Magazine extends Media { Magazine(String title, int lateDays) { super(title, lateDays); } double lateFee() { return lateDays * 1.0; } } public class MediaDemo { public static void main(String[] args) { Media[] items = { new Book("Java Book", 3), new DVD("Movie DVD", 2), new Magazine("Tech Magazine", 4) }; double total = 0;</pre>
------	---

```
for (Media item : items) {  
    double fee = item.lateFee();  
    System.out.println(item.title + ": Rs. " + fee);  
    total += fee;  
}  
  
System.out.println("Total late fee: Rs. " + total);  
}  
}
```

Output	Java Book: Rs. 6.0 Movie DVD: Rs. 10.0 Tech Magazine: Rs. 4.0 Total late fee: Rs. 20.0
---------------	---

Program: 5.4	Project Work: MiniBank — Account Types A real bank has different account types. Building on the single Account class, this lab makes Account abstract and adds three subclasses — SavingsAccount, CurrentAccount and FixedDepositAccount — each with its own interest and withdrawal rules chosen automatically by polymorphism. This lets the rest of MiniBank treat every account through one common type. 1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount). 2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance. 3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit. 4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked). 5. Call super(...) from each subclass constructor to initialise the inherited fields. 6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially. Expected output: The program prints 4.0 for the savings account, 0.0 for the current account and 7.0 for the fixed deposit.
---------------------	---

Code	<pre> abstract class Account { String accountNumber; String ownerName; long balance; Account(String accountNumber, String ownerName, long balance) { this.accountNumber = accountNumber; this.ownerName = ownerName; this.balance = balance; } abstract double interestRate(); abstract boolean canWithdraw(long amount); } class SavingsAccount extends Account { long minBalance; SavingsAccount(String number, String owner, long balance, long minBalance) { super(number, owner, balance); this.minBalance = minBalance; } double interestRate() { return 4.0; } boolean canWithdraw(long amount) { return amount > 0 && balance - amount >= minBalance; } } class CurrentAccount extends Account { long overdraftLimit; CurrentAccount(String number, String owner, long balance, long overdraftLimit) { super(number, owner, balance); this.overdraftLimit = overdraftLimit; } double interestRate() { return 0.0; } boolean canWithdraw(long amount) { return amount > 0 && balance - amount >= -overdraftLimit; } } class FixedDepositAccount extends Account { </pre>
------	---

```
FixedDepositAccount(String number, String owner, long balance) {
    super(number, owner, balance);
}

double interestRate() {
    return 7.0;
}

boolean canWithdraw(long amount) {
    return false;
}
}

public class MiniBank {
    public static void main(String[] args) {
        Account[] accounts = {
            new SavingsAccount("AC0001", "Riya", 10000, 2000),
            new CurrentAccount("AC0002", "Amit", 5000, 3000),
            new FixedDepositAccount("AC0003", "Jay", 20000)
        };

        for (Account account : accounts) {
            System.out.println(account.accountNumber + " interest rate: "
                + account.interestRate() + "%");

            if (account instanceof SavingsAccount savings) {
                System.out.println("Savings withdrawal allowed: "
                    + savings.canWithdraw(3000));
            }
        }
    }
}
```

Output	AC0001 interest rate: 4.0% Savings withdrawal allowed: true AC0002 interest rate: 0.0% AC0003 interest rate: 7.0%
---------------	--

	Practical 6: INTERFACES, PACKAGES AND JAR
Program: 6.1	Remote control An interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

Code	<pre>interface Switchable { void on(); void off(); default void toggle() { off(); on(); } } class Fan implements Switchable { public void on() { System.out.println("Fan is ON"); } public void off() { System.out.println("Fan is OFF"); } } class Light implements Switchable { public void on() { System.out.println("Light is ON"); } public void off() { System.out.println("Light is OFF"); } } @FunctionalInterface interface SwitchRule { boolean canSwitchOn(String device, int hour); } public class RemoteControl { public static void main(String[] args) { Switchable[] devices = {new Fan(), new Light()}; for (Switchable device : devices) { device.toggle(); } SwitchRule rule1 = new SwitchRule() { public boolean canSwitchOn(String device, int hour) { return hour >= 6 && hour <= 22; } }; SwitchRule rule2 = (device, hour) -> hour >= 6 && hour <= 22;</pre>
-------------	---

	<pre>System.out.println(rule1.canSwitchOn("Fan", 10)); System.out.println(rule2.canSwitchOn("Light", 23)); } }</pre>
--	--

Output	Fan is OFF Fan is ON Light is OFF Light is ON true false
---------------	---

Program: 6.2	Notification senders A functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.
---------------------	--

Code	<pre>@FunctionalInterface interface Notifier { void send(String message); } interface Urgent { } class UrgentNotifier implements Notifier, Urgent { private Notifier notifier; UrgentNotifier(Notifier notifier) { this.notifier = notifier; } public void send(String message) { notifier.send(message); } } public class NotificationDemo { public static void main(String[] args) { Notifier email = message -> System.out.println("Email: " + message); Notifier sms = message -> System.out.println("SMS: " + message); Notifier[] senders = { new UrgentNotifier(email), sms }; String message = "Your payment is complete"; for (Notifier sender : senders) { sender.send(message); if (sender instanceof Urgent) { sender.send(message); } } } }</pre>
-------------	--

Output	Email: Your payment is complete Email: Your payment is complete SMS: Your payment is complete
---------------	---

Program: 6.3	(For Advanced Learners – home) Discount engine Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.
---------------------	--

Code

```
import java.util.*;

@FunctionalInterface
interface DiscountRule {
    double apply(double price);
}

public class DiscountEngine {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double[] prices = {1000, 2000, 3000};

        System.out.println("1. 10% discount");
        System.out.println("2. 20% discount");
        System.out.println("3. No discount");
        System.out.print("Choose rule: ");

        int choice = sc.nextInt();

        DiscountRule rule = switch (choice) {
            case 1 -> price -> price * 0.90;
            case 2 -> price -> price * 0.80;
            default -> price -> price;
        };

        double total = 0;

        for (double price : prices) {
            double finalPrice = rule.apply(price);
            System.out.printf("%.2f -> %.2f%n", price, finalPrice);
            total += finalPrice;
        }

        System.out.printf("Total: %.2f%n", total);
        sc.close();
    }
}
```

Output	<pre>1. 10% discount 2. 20% discount 3. No discount Choose rule: 1 1000.00 -> 900.00 2000.00 -> 1800.00 3000.00 -> 2700.00 Total: 5400.00</pre>
---------------	--

Program: 6.4	Project Work: MiniBank — Interfaces, Packages and Runnable JAR With several classes now in place, this lab gives MiniBank a clean structure. You express account capabilities as interfaces, organise the code into model, util and service packages, and build a runnable JAR — the form in which the finished application will actually be delivered and run. 1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it. 2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance. 3. Create a functional interface named WithdrawRule with a single method boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression. 4. Create a marker interface named Premium (an interface with no methods). 5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import. 6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar. Expected output: Both versions return the same allow() result for the same inputs; java -jar minibank.jar launches the menu from Practical 1.
---------------------	--

Code	<pre>import static java.lang.System.out; interface Transactable { void deposit(long amount); boolean withdraw(long amount); } interface InterestBearing { double interestRate(); long getBalance(); default double yearlyInterest() { return getBalance() * interestRate() / 100; } } @FunctionalInterface interface WithdrawRule { boolean allow(Account account, long amount); } interface Premium { } class Account implements Transactable, InterestBearing, Premium { private String accountNumber; private long balance; Account(String accountNumber, long balance) { this.accountNumber = accountNumber; this.balance = balance; } public void deposit(long amount) { if (amount > 0) { balance += amount; } } public boolean withdraw(long amount) { if (amount > 0 && amount <= balance) { balance -= amount; return true; } return false; } public double interestRate() { return 4.0; } }</pre>
-------------	--

```
public long getBalance() {
    return balance;
}

public String getAccountNumber() {
    return accountNumber;
}
}

public class MiniBank {
    public static void main(String[] args) {
        Account account = new Account("AC0001", 5000);

        WithdrawRule rule1 = new WithdrawRule() {
            public boolean allow(Account account, long amount) {
                return amount > 0 && amount <= account.getBalance();
            }
        };

        WithdrawRule rule2 = (acc, amount) ->
            amount > 0 && amount <= acc.getBalance();

        out.println("Account: " + account.getAccountNumber());
        out.println("Balance: " + account.getBalance());
        out.println("Yearly interest: " + account.yearlyInterest());
        out.println("Anonymous rule: " + rule1.allow(account, 1000));
        out.println("Lambda rule: " + rule2.allow(account, 1000));

        account.deposit(1000);
        account.withdraw(500);

        out.println("Updated balance: " + account.getBalance());
        out.println("Premium account: " + (account instanceof Premium));
    }
}
```

Output	Account: AC0001 Balance: 5000 Yearly interest: 200.0 Anonymous rule: true Lambda rule: true Updated balance: 5500 Premium account: true
---------------	---